

Architecture Agentique pour l'Extraction de Connaissances sur Code Propriétaire (Envision)

Soutenance PSC
Mardi 19 mai 2026

Plan

① Introduction

Le langage Envision

② État de l'art

③ Architecture

Présentation Globale

Nos cinq outils

Query Transformers : optimisation du RAG

④ Context Engineering

⑤ Benchmarking

⑥ Résultats

⑦ Conclusion

Table des matières

① Introduction

Le langage Envision

② État de l'art

③ Architecture

Présentation Globale

Nos cinq outils

Query Transformers : optimisation du RAG

④ Context Engineering

⑤ Benchmarking

⑥ Résultats

⑦ Conclusion

Le Défi des DSL Industriels

L'angle mort des Grands Modèles de Langage

- **Succès des LLM sur les langages standards** : Sur Python ou Javascript, des outils comme Copilot agissent en assistants omniscients (surabondance de données d'entraînement publiques).
- **La réalité industrielle** : De nombreuses entreprises fondent leur compétitivité sur des **Domain Specific Languages (DSL)** propriétaires.
- **Le cas d'Envision (Lokad)** : Un langage spécialisé pour l'optimisation de la supply chain.

Problématique

Les LLMs du marché (GPT-4, Claude, Mistral) n'ont quasiment jamais rencontré de code Envision, menant à des **hallucinations massives** de syntaxe et de structure.

Objectifs : De la recherche simple à l'Agentique

Nous ne cherchons pas seulement à construire un moteur de recherche, mais un **système agentique hybride** capable de :

- 1 **Classifier l'intention** : Distinguer une question conceptuelle d'une recherche chirurgicale d'identifiant.
- 2 **Orchestrer divers outils** : Décider de manière autonome quel outil (RAG, Grep, Graphe de dépendances...) appeler en fonction des preuves déjà collectées.
- 3 **Itérer et s'auto-corriger** : Ajuster sa stratégie en continu via une boucle de feedback basée sur *LangGraph* pour corriger les erreurs de parcours.

Les Défis du Projet Envision

1. Volumétrie et couplage

Un compte client typique est composé de près d'une centaine de scripts complexes interconnectés, de fichiers d'entrées/sorties et de modules d'optimisation numérique.

2. Documentation lacunaire

Le code est la seule source de vérité fiable. Les dépendances de fichiers sont souvent construites par interpolation de chaînes (*paths variables*), invisibles à l'analyse statique classique.

3. Besoin d'évaluation objective

Nécessité de construire un protocole de benchmarking automatisé (*LLM-as-a-Judge*) pour mesurer les progrès de nos prototypes.

Lokad et Envision : Le Terrain Applicatif

- **Spécialisation** : Plateforme logicielle spécialisée dans l'optimisation quantitative de la supply chain (prévisions de ventes, réapprovisionnement, allocation de stocks).
- **Une logique orientée Batch et Décision** : Les scripts Envision lisent des fichiers plats d'entrée (catalogues, commandes), effectuent des traitements et produisent :
 - Des fichiers nettoyés (/Clean/...)
 - Des scores opérationnels et suggestions d'achat
 - Des dashboards décisionnels directement exploitables
- **Complexité à l'échelle** : Plus le corpus de scripts grandit (modules interconnectés, imports transverses), plus l'exploration manuelle par un développeur (ou un modèle) devient difficile.

Envision : Un DSL « 3-en-1 » pour la Supply Chain

Envision combine trois dimensions habituellement séparées :

1. Tabulaire

Déclaratif (SQL-like)

- Manipulation de tables
- Agrégations et jointures
- Séries temporelles

2. Procédural

Contrôlé (Python-like)

- Iterations & flux bornés
- Fonctions stateless (def pure, def const pure)
- Fonctions stateful (def process)

3. Restitution

Visualisation

- Dashboarding natif
- show table
- show plot...

Une application batch complète en un seul langage.

Table des matières

① Introduction

Le langage Envision

② État de l'art

③ Architecture

Présentation Globale

Nos cinq outils

Query Transformers : optimisation du RAG

④ Context Engineering

⑤ Benchmarking

⑥ Résultats

⑦ Conclusion

Émergence des LLM

- Années 80 : premiers succès des réseaux de neurones
- 2017 : révolution des *Transformers*
 - article *Attention is all you need* [1]
 - Application au NLP
- Agentification
- Limites : hallucinations

Context Engineering

- Extension du Prompt Engineering pour les agents IA
- Objectif :
 - optimiser les informations fournies au modèle
- Difficulté :
 - trop d'informations = bruit
 - pas assez d'informations = capacités limitées
- Techniques utilisées :
 - filtrage dynamique
 - recherche lexicale (grep, regex)

Retrieval Augmented Generation (RAG)

- Principe :
 - récupérer des documents pertinents
 - les fournir au LLM avant génération
- Recherche sémantique :
 - représentation vectorielle (*embeddings*)
 - proximité sémantique entre textes
- Améliorations plus avancées (RAPTOR, HyDe...)

Table des matières

① Introduction

Le langage Envision

② État de l'art

③ Architecture

Présentation Globale

Nos cinq outils

Query Transformers : optimisation du RAG

④ Context Engineering

⑤ Benchmarking

⑥ Résultats

⑦ Conclusion

Préparation des données : le problème du chunking

- Le code source possède une structure hiérarchique forte
- Les méthodes classiques de chunking sont inadaptées :
 - découpage arbitraire
 - perte du contexte logique
- Risques :
 - séparation d'une fonction et de sa signature
 - perte des déclarations importantes
 - compréhension dégradée du code
- Besoin :
 - conserver des blocs cohérents sémantiquement

EnvisionParser

Extraction sémantique du code Envision

Objectif

Transformer les scripts `.nvn` en blocs logiques exploitables par le pipeline RAG, tout en conservant les dépendances et le contexte structurel du code.

1. Identification des Blocs

Analyse ligne par ligne

- Détection des blocs READ, TABLE, SHOW, etc.
- Gestion des structures multilignes

2. Extraction des Dépendances

Analyse sémantique

- Extraction des tables utilisées
- Filtrage des built-ins Envision
- Détection des variables définies

Script Envision → 6078 CodeBlock → Chunker

EnvisionChunker

Segmentation sémantique optimisée pour le RAG

Principe

Construire des chunks cohérents sémantiquement afin de préserver la logique du code tout en respectant les contraintes de taille des modèles d'embedding.

1. Construction des Chunks

Regroupement adaptatif

- Limite configurable (ex : 512 tokens)
- Respect des frontières logiques (sections ///**=====**)
- Découpage des gros blocs

2. Préservation du Contexte

Gestion du contexte local

- Overlap entre chunks adjacents
- Conservation des métadonnées
- Suivi des dépendances inter-blocs

6078 CodeBlock → 1084 CodeChunk → Embeddings + FAISS

RAPTOR

Retrieval hiérarchique par structure arborescente

Principe général

RAPTOR [2] construit une représentation hiérarchique des documents en regroupant récursivement les chunks en clusters, chacun résumé pour créer plusieurs niveaux d'abstraction.

1. Construction de l'arbre

Organisation hiérarchique

- Embeddings des chunks initiaux
- Clustering (UMAP + GMM)
- Résumés récursifs des clusters

2. Index multi-niveaux

Plusieurs granularités d'accès

- Niveau global (résumés)
- Niveau fin (chunks sources)

Chunks → Clusters → Résumé hiérarchique → Index RAPTOR

Pipeline actuelle

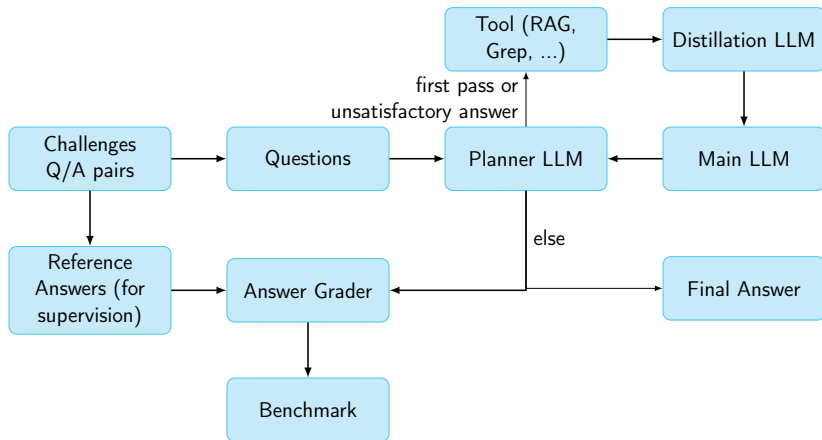


Figure 1 – Pipeline Agentique actuelle

RAG Tool : Approche vectorielle pure

1. Tokenisation

Découpage du texte brut en unités élémentaires indexées :

"Le chat dort" → Token 1 : 412 Token 2 : 1895 Token 3 : 34

2. Embedding

Transformation d'un token discret x (représenté par son vecteur *one-hot*) en un vecteur dense v via la matrice de projection E :

$$v = E \cdot x_{one-hot} \quad \text{où} \quad E \in \mathbb{R}^{d \times |V|}$$

- d : dimension de l'espace latent (ex : 768, 1536).
- $|V|$: taille totale du vocabulaire du tokenizer.

RAG Tool : Approche vectorielle pure

3. Transformer

Forward-pass de la séquences d'embeddings dans un *Transformer* pour obtenir l'embedding d'un ensemble de mots

Cosine Similarity

Mesure de proximité sémantique par alignement directionnel :

$$\text{sim}(u, v) = \cos(\theta) = \frac{u \cdot v}{\|u\| \|v\|}$$

Interprétation par l'angle :

- $\theta \approx 0^\circ$ ($\cos \approx 1$) : Concepts quasi-identiques.
- $\theta \approx 90^\circ$ ($\cos \approx 0$) : Indépendance sémantique (orthogonalité).

RAG Tool : Recherche *sparse*, algorithme BM25 [3]

Formule de pertinence fréquentielle

$$\text{Score}(D, Q) = \sum_{q \in Q} \text{IDF}(q) \cdot \frac{f(q, D) \cdot (k_1 + 1)}{f(q, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdI}}\right)}$$

L'équation orchestre trois composantes fondamentales :

- **IDF(q)** (*Rareté*) : Favorise les termes discriminants et réduit à zéro l'impact des mots trop communs du code.
- **Paramètre $k_1 \in [1.2, 2.0]$** (*Saturation*) : Impose une limite asymptotique pour éviter qu'un terme répété en boucle n'écrase le score.
- **Paramètre $b \approx 0.75$** (*Normalisation*) : Pénalise les fichiers longs ($|D| \gg \text{avgdI}$)

RAG Tool : Approche hybride avec Qdrant

Portée \ Méthode	Dense (Sémantique)	Sparse (BM25)
Global	Flux 1 : Capture les concepts, la logique métier et les intentions globales.	Flux 2 : Capture la syntaxe exacte, les noms de variables et les identifiants techniques.
Filtré (Sources)	Flux 3 (Boosté) : Recherche conceptuelle limitée aux fichiers cibles.	Flux 4 (Boosté) : Recherche de mots-clés exacte limitée aux fichiers cibles.

Reciprocal Rank Fusion [4]

$$RRF(d) = \sum_{r \in R} \frac{1}{k + r(d)}$$

où $r \in R$ sont les flux de recherche et $k = 60$ est une constante

Grep Tool : Recherche lexicale exacte pilotée par la structure

- **Le besoin** : Le RAG sémantique est parfois trop flou pour trouver un identifiant précis, un chemin exact ou un littéral de code.
- **Le principe** : Une recherche textuelle exacte (Regex), mais **guidée par la structure syntaxique** du code Envision parsé (IMPORT, READ, WRITE, SHOW, EXPORT).
- **Trois paramètres de filtrage chirurgical** :
 - ① **pattern** : Le motif recherché (ex : "StockEvo1").
 - ② **sources** : Restriction à un sous-dossier ou à un script spécifique.
 - ③ **block_type** : Filtre syntaxique pour ne chercher que dans les blocs déclarés (ex : uniquement les READ ou les SHOW).

Grep Tool : Résolution récursive des chemins

La complexité des interpolations de chaînes

Dans Envision, de nombreux chemins sont dynamiques : `const`

```
inputPath = "{testStorage}Clean/"  
read "{inputPath}Items.ion" as Items
```

- **Le problème** : Une recherche classique sur `"/Clean/Items.ion"` échoque car la chaîne n'apparaît jamais textuellement en entier sur une ligne.
- **La solution** : Implémentation d'un résolveur récursif à la volée (*runtime backtracking*) qui remonte la chaîne de définitions de variables pour reconstituer le chemin réel.
- **Le retour contextuel** : L'outil renvoie le bloc de code correspondant tronqué autour du motif avec une fenêtre de lignes configurables, respectant le budget de tokens du modèle.

Graph Tool : Mémoire structurelle précompilée

- **Le besoin** : RAG (sémantique) et Grep (lexique) ne permettent pas de comprendre l'architecture globale d'un projet complexe (dépendances inter-scripts, flux de données).
- **Le concept** : Reconstruire statiquement les dépendances du projet sous la forme d'un **graphe de dépendances relationnelles** en résolvant des globs (*) et des interpolations de chemins
- **Les 5 types de nœuds** :
 - folder (dossier), script (fichier Envision .nvn), data_file (.ion, .csv, .xlsx), table (table déclarée), function (process défini).
- **Les 6 types d'arêtes** : imports, reads, writes, defines, contains, sibling.

Graph Tool : Processus de Construction

1. Processus

Analyse Statique

Lecture des scripts
.nvn

Résolution

Calcul des variables et
globs (*)

Relations

Chainage hiérarchique
et voisins (sibling)

2. Structure (RAM)

Network Container

nodes : Dict[ID,
Node]
edges : List[Edge]

Node Object

Id, Type, Contenu
(code, table, func),
Métadonnées

Edge Object

Source, Target, Type

3. Stockage (Cache)

network.json

Persistance à plat
(sommets/arêtes
sérialisés)

metadata.json

Métadonnées globales
et statistiques du build

Audit

Rapport de résolution
des globs/placeholders

Scripts .nvn → **Network Builder** → Network → **JSON sérialisé**

Graph Tool : Statistiques et Apport pour l'Agent

Statistiques réelles du dépôt client étudié

Le graphe généré à partir de notre corpus contient :

- **300 nœuds** (64 scripts, 73 fichiers de données, 96 tables, 17 fonctions, 50 dossiers).
- **781 arêtes** (55 imports, 270 reads, 63 writes, 113 defines).
- **API et outils exposés** : L'agent interroge le graphe via des fonctions dédiées :
 - tree (structure)
 - node (métadonnées)
 - neighbors (voisins)
 - edges (relations globales)
- **Apport pour le LLM** : Fournit une mémoire structurelle relationnelle fiable.

Graph Tool : Cas d'utilisation concrets

- **Cas 1 : Découverte d'arborescence ciblée (tree)**
`tree(path="/1. utilities", domain="scripts", max_depth=2)`
→ *Retourne l'arborescence des modules utilitaires limitée à deux niveaux.*
- **Cas 2 : Ligne de vie d'un fichier de données (neighbors)**
`neighbors(node_id="/Clean/Orders.ion", direction="incoming", relation_type="reads")`
→ *Retourne la liste de tous les scripts Envision lisant le fichier de commandes.*
- **Cas 3 : Cartographie globale des importations (edges)**
`edges(relation_type="imports")`
→ *Filtre et affiche l'ensemble des modules importés à l'échelle du projet.*

TreeTool

- Un outil (`tree_tool`) fournit une représentation textuelle de l'arborescence des fichiers.
- Format Linux `tree` (`|`, dossiers avec `/`)
- Deux niveaux d'usage :
 - `_kickoff_prompts()` : arborescence complète
 - `_continuation_prompts()` : version réduite
- Réduction automatique de la profondeur ou largeur pour respecter les limites de tokens.
- → Outil est rarement appelé par le planner.

Script Finder Tool

Situation initiale

Dump horizontal de fichiers

- 68035.nvn
- 68036.nvn
- 68037.nvn
- ...

mapping.txt



script_finder_tool



Situation finale

Arborescence virtuelle

- [PATH_1] [NOM_1].nvn
- [PATH_2] [NOM_2].nvn
- [PATH_3] [NOM_3].nvn
- ...

Attention : Utilisation intensive de Tokens = saturation du contexte !

Interface *OpenAI-compatible*

- Outils exposés au modèle via une syntaxe standardisée *OpenAI* prise en charge par les API.
- Format typique fourni au modèle :

```
{ "type": "function",  
  "function": {  
    "name": "tool_name",  
    "description": "Ce que fait le tool",  
    "parameters": {  
      "type": "object",  
      "properties": { /* arguments */ },  
      "required": [ /* arguments obligatoires */ ]  
    }  
  }  
}
```

Cela permet de guider le modèle dans le choix des outils.

Query Transformers

- Objectif : Pallier l'écart sémantique question - réponse
- Donc modification de la réponse AVANT retrieval
- Deux approches :
 - RAG Fusion : Elargir le spectre sémantique
 - HyDE : Générer des réponses types

Rappel de la Reciprocal Rank Fusion [4]

$$RRF(d) = \sum_{r \in R} \frac{1}{k + r(d)}$$

où $r \in R$ sont les flux de recherche et $k = 60$ est une constante

Table des matières

① Introduction

Le langage Envision

② État de l'art

③ Architecture

Présentation Globale

Nos cinq outils

Query Transformers : optimisation du RAG

④ Context Engineering

⑤ Benchmarking

⑥ Résultats

⑦ Conclusion

Les instructions de base

Fichier Base_instructions.txt

- Fourni dans le *kickoff* et les *continuation prompts*
- Informations sur la syntaxe Envision
- Consignes/Philosophie générale

Objectifs :

- Eviter la répétition inutile d'instructions
- Cohérence du raisonnement
- Pallier un manque de connaissance d'Envision

La distillation

Objectif :

- Convertir des résultats bruts en faits concis
- Conserver une traçabilité des résultats

Fonctionnement :

- Tout résultat d'un tool est gardé de côté
- Lazy Distillation : Distillation durant la phase de planning, pas à la récupération par l'outil. On évite de distiller abusivement.

Extrait de prompt

"Analyze the items above. Extract ONLY the facts that directly and specifically help answer the Query or the Thought. Be selective — a single precise fact is better than five vague ones. If two items say the same thing, merge them into one fact. If an item is irrelevant to the query, ignore it entirely.

Prior Evidence Tool et types de faits

- Objectif : réaccéder à des résultats précédemment trouvés sans nouvel appel aux tools
- Possibilité de :
 - Combiner des résultats de différents retrievals
 - Accéder à de l'info de questions précédentes

RetrievalResult

chunk
score
rank

KnowledgeElement

fact
tool
query
evidence_ids

ActionLog

step
tool
query :
evidence_ids
query :
thought
parameters
outcome_summary

Le stockage du contexte

Nom	Type de Donnée	Objectif	Source
Knowledge Bank	List[Knowledge Element]	Faits distillés donnés au <i>Planner</i>	<i>Distillation Tool</i>
Accumulated Evidence	Dict[Id → Retrieval Result]	Résultats bruts pour le <i>prior_evidence_tool</i>	<i>Distillation Tool</i>
Execution History	List[ActionLog]	Appels précédents donnés au <i>Planner</i>	<i>Tools</i>

Synthèse des structures de données du projet

les motivations derrière :

Les motivations derrière cette implémentation sont multiples :

- **Efficacité en termes de tokens** : Réduction de la taille du contexte en le limitant aux faits pertinents. Appels répétitifs supprimés
- **Traçabilité du raisonnement** : Visibilité sur le cheminement du raisonnement via le système d'ActionLogs et d'Ids liés aux faits
- **Mode conversationnel avec mémoire** : Exploitation possible des questions précédentes.

Table des matières

① Introduction

Le langage Envision

② État de l'art

③ Architecture

Présentation Globale

Nos cinq outils

Query Transformers : optimisation du RAG

④ Context Engineering

⑤ Benchmarking

⑥ Résultats

⑦ Conclusion

Le Défi de l'Évaluation

Pourquoi les métriques classiques échouent

Approche Standard

Similarité Cosinus (*Embeddings*)

- Calcule la ressemblance de la forme textuelle
- Mesure la proximité des mots-clés utilisés
- Tolère les approximations logiques

Exigence du DSL Propriétaire

Rupture Logique Factuelle

- Nécessite une exactitude déterministe
- Un seul chiffre erroné invalide complètement la réponse

Stratégie d'Évaluation Hybride

Une approche bimodale selon la nature du problème

1. Question Sémantique

LLM-as-a-Judge

- Arbitrage par un modèle tiers indépendant
- Analyse critique de la cohérence logique du fond
- Pénalisation de l'illusion d'une bonne rédaction

2. Question Structurale

Expressions Régulières (Regex)

- Validation déterministe pour les données exactes
- Score strictement binaire : 0 ou 1
- Coût de calcul nul et certitude mathématique

Le Dual Cross-Encoder

Saisir les ruptures logiques par attention croisée

Principe de l'Attention Croisée

Contrairement aux architectures classiques qui comparent des vecteurs isolés, le Cross-Encoder fait interagir chaque mot de la question avec chaque mot de la réponse au sein du Transformer.

1. Filtre de Cohérence Logique

Modèle DeBERTa v3 (NLI)

- Spécialisé dans l'inférence
- Détection des contradictions
- Effet couperet : score ramené à 0 en cas d'erreur majeure

2. Évaluation de la Pertinence

Modèle Jina Transformer

- Analyse continue de l'alignement
- Mesure l'utilité contextuelle
- Valide la précision face au besoin de l'utilisateur

Calcul du Score

Logique de décision du Dual Cross-Encoder

1. Filtre Sanction (DeBERTa v3)

Vérification de la cohérence logique (NLI) :

$$\text{Si } S_{\text{contradiction}} > 0.5 \implies S_{\text{total}} = 0$$

Justification : Une erreur factuelle invalide toute la réponse.

2. Fusion Sémantique et Logique

En l'absence de contradiction, calcul de la moyenne pondérée :

- **60%** : Cohérence (*Entailment* DeBERTa)
- **40%** : Sémantique (*Jina* avec normalisation σ)

$$S_{\text{total}} = 0.6 \cdot S_{\text{entailment}} + 0.4 \cdot \sigma(S_{\text{Jina}})$$

$$\sigma(x) = \frac{1}{1+e^{-x}} \text{ (Normalisation par sigmoïde du score Jina)}$$

Constitution du Dataset de Test

27 scénarios issus de l'historique réel de Lokad

Corpus de validation : 27 paires Questions-Réponses critiques

1. Requêtes Structurelles (7 cas)

Vérification de la navigation technique et de la syntaxe.

- *Exemple* : « Quels scripts lisent le fichier /Clean/Items.ion ? »
- Objectif : Valider l'activation des outils graph ou grep.

2. Requêtes Conceptuelles (20 cas)

Compréhension de la logique métier Supply Chain.

- *Exemple* : « Où est définie la logique d'actualisation du stock ? »
- Objectif : Tester le raisonnement multi-étapes et la qualité du RAG.

Table des matières

① Introduction

Le langage Envision

② État de l'art

③ Architecture

Présentation Globale

Nos cinq outils

Query Transformers : optimisation du RAG

④ Context Engineering

⑤ Benchmarking

⑥ Résultats

⑦ Conclusion

Répartition des appels entre les outils

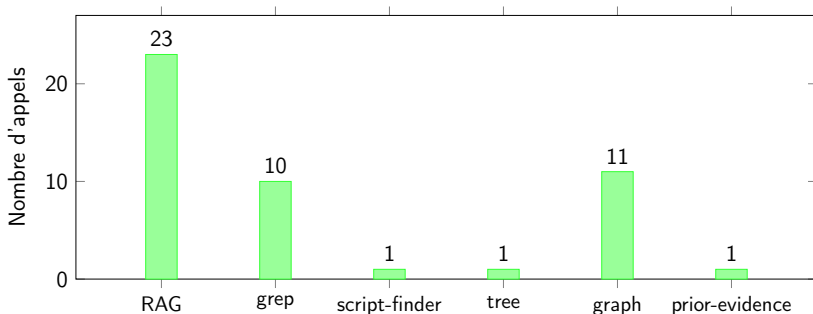


Figure 2 – Fréquence d'utilisation des outils par l'agent.

Décomposition du temps d'inférence

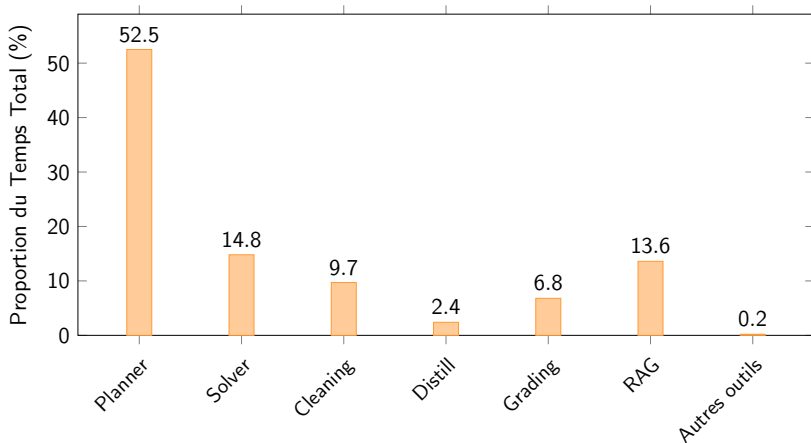


Figure 3 – Répartition du temps d'exécution par composant et rôle LLM pour une durée totale de 30m 12s.

Étude d'ablation sur l'outil RAG

Configuration	Score (sur 20)	Δ Score	Temps (s)	Δ Temps (s)
FAISS	8	-6	0.47	-246.70
Qdrant-Hybride	9	-5	2.65	-244.52
Qdrant+Jina (Réf.)	14	—	247.17	—
No-Summary	6	-8	243.05	-4.12
Fusion	12	-2	519.39	+272.22
HyDE	12	-2	709.18	+462.01

Table 1 – Étude d'ablation : évaluation de l'impact des différentes stratégies par rapport à l'architecture de référence.

Table des matières

① Introduction

Le langage Envision

② État de l'art

③ Architecture

Présentation Globale

Nos cinq outils

Query Transformers : optimisation du RAG

④ Context Engineering

⑤ Benchmarking

⑥ Résultats

⑦ Conclusion

Limites & Recul Critique

Les leçons du terrain et pistes d'évolution

Limites Identifiées

- **Sensibilité du RAG** : Risque d'aiguillage erroné sur des termes polysémiques (ex : « *croissance* »).
- **Biais de Dépôt** : Risque d'overfitting du prompt, testé sur un compte client unique.

Pistes d'Amélioration

- **Granularité** : Étendre le graphe relationnel jusqu'à l'arborescence des colonnes de données.
- **Garde-fou** : Filtrage final rejetant toute conclusion privée de source explicite.

Conclusion Générale

Succès de l'approche agentique

La mise en œuvre d'un graphe d'états non linéaire a permis de valider la faisabilité d'un assistant intelligent capable de naviguer dans la complexité du langage Envision.

Collaboration industrielle

- Immersion dans l'écosystème Lokad
- Dialogue constant avec les experts métier
- Confrontation du modèle aux exigences réelles de la supply chain

Synergie d'équipe et Bilan humain

- Coordination sur une architecture modulaire
- Montée en compétence
- Rigueur scientifique

References I

- [1] A. VASWANI et al. « Attention Is All You Need ». In : *Advances in Neural Information Processing Systems (NeurIPS)*. 2017.
- [2] P. SARTHI et al. « RAPTOR : Recursive Abstractive Processing for Tree-Organized Retrieval ». In : *International Conference on Learning Representations (ICLR)*. 2024. URL : <https://arxiv.org/abs/2401.18059>.
- [3] S. ROBERTSON et H. ZARAGOZA. « The Probabilistic Relevance Framework : BM25 and Beyond ». In : *Foundations and Trends in Information Retrieval* (2009). DOI : 10.1561/1500000019. URL : <https://doi.org/10.1561/1500000019>.

References II

- [4] G. V. CORMACK, C. L. A. CLARKE et S. BÜTTCHER. « Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods ». In : *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval (SIGIR)*. 2009. URL : <https://dl.acm.org/doi/10.1145/1571941.1572114>.

Annexe 1 : Formule IDF

IDF : quantification de la rareté

$$\text{IDF}(q) = \ln \left(\frac{N - n(q) + 0.5}{n(q) + 0.5} + 1 \right)$$

où N représente le nombre total de documents dans la collection, et $n(q)$ est le nombre de documents contenant le terme q .

Annexe 2 : Envision – Hello World & Affichage

Envision compile directement en tableaux de bord web réactifs. Pas de `print()` CLI classique, l'équivalent est l'affichage de composants via `show`.

Syntaxe de base (`show label`)

```
// Hello World sur le tableau de bord
show label "Hello World!"

x = 42
show label "La valeur est {x}"
```

Annexe 2 : Envision – Définition de Fonctions

Envision sépare strictement les fonctions pures (calculs déterministes sans effet de bord) des fonctions de processus (maintien d'état pour les simulations temporelles).

Fonction Pure (def pure)

```
// Déclaration d'une fonction de somme simple
def pure sum(a: number, b: number) with
  return a + b

res = sum(10, 32)
show label "Somme = {res}"
```


Annexe 2 : Envision – Tableaux en ligne

Pour manipuler des données locales ou définir des correspondances statiques, Envision utilise le concept de *Table Comprehension*.

Création de table inline (table T = with)

```
table T = with
  [| as X, as Y |] // Déclaration des en-têtes de colonnes
  [| 1, 10 |]      // Lignes de données associées
  [| 2, 25 |]
  [| 3, 40 |]
```

Annexe 2 : Envision – Entrées / Sorties (I/O)

La persistance des données s'effectue via les instructions `write` et `read`.
Le format binaire propriétaire `.ion` est optimisé pour les volumes logistiques massifs.

Stockage atomique et lecture de schéma

```
// 1. Écriture atomique dans un fichier .ion
write T as "/Clean/TutoData.ion" with
  X = T.X
  Y = T.Y

// 2. Lecture (Récupération) et définition du schéma
read "/Clean/TutoData.ion" as T2 with
  X : number
  Y : number
```

Annexe 2 : Envision – Modification & Graphiques

Les modifications de données s'effectuent par des opérations vectorielles.
Les graphiques et tableaux s'insèrent sur le dashboard à l'aide de coordonnées de grille (ex : a1c3).

Calcul vectoriel et tracé de courbe $f(X) = Y$

```
// 1. Calcul vectoriel sur colonne  
T2.Y_mod = T2.Y + 5  
  
// 2. Rendu de tableau et de courbe plot  
show table "Données Modifiées" a1c3 with T2.X, T2.Y_mod  
show plot "Courbe d'Evolution Y = f(X)" d1f3 with  
  T2.X as "X"  
  T2.Y_mod as "Y"
```

Annexe 3 : Performance en fonction des modèles utilisés

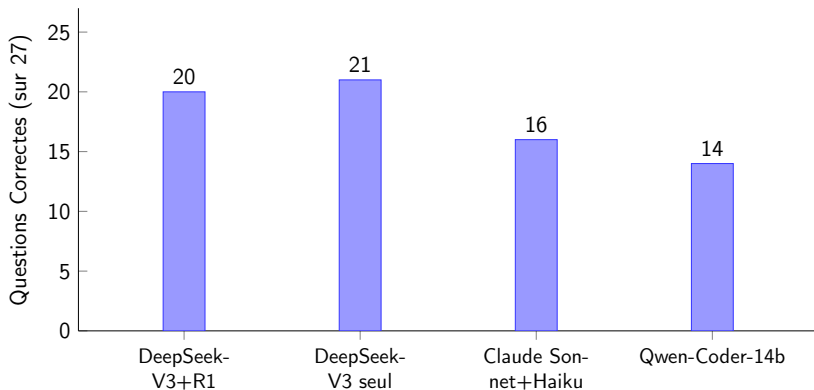


Figure 4 – Performance comparative des LLM sur le benchmark de 27 questions